

LAB 3: Uniform Cost Search

Now the edges have costs. Find the cheapest path from A to D.

A → B	1	A → C	4	B → D	5	B → C	2	C → D	1
-------	---	-------	---	-------	---	-------	---	-------	---

There are exactly three routes: A → B → D costs 6,

A → C → D costs 5, A → B → C → D costs 4.

Answer this: the cheapest route is the one with the most hops. BFS would return a 2-hop route costing 6. Why must the goal be tested when a path is popped, not when it is generated?

This activity is about understanding how Uniform Cost Search (UCS) finds the lowest-cost path in a weighted graph.

Uniform Cost Search (UCS) is a graph search algorithm that always expands the path with the smallest cumulative cost from the start node. Unlike BFS, UCS does not care about the number of edges (hops); it only considers the total path cost.

A priority queue is used to store paths, where the path with the lowest cost is removed first.

Graph

Edge	Cost
A → B	1
A → C	4
B → D	5
B → C	2
C → D	1

Possible Routes

Route	total cost
A → B → D	6
A → C → D	5
A → B → C → D	4 (Cheap)

Lab 3: UCS (uniform cost search)

```
graph = {'A': [('B', 1), ('C', 4)],  
         'B': [('D', 5), ('C', 2)],  
         'C': [('D', 1)],  
         'D': []}
```

```
def ucs(start, goal):
```

```
    queue = [0, [start]]
```

```
    while queue:
```

```
        queue.sort()
```

```
        cost, path = queue.pop(0)
```

```
        if path[-1] == goal:
```

```
            return cost, path
```

```
        for n, w in graph[path[-1]]:
```

```
            queue.append((cost + w, path + [n]))
```

```
    return None
```

```
print("UCS:", ucs('A', 'D'))
```

//_

(Output): UCS: (4, ['A', 'B', 'C', 'D'])

- The cheapest path is $A \rightarrow B \rightarrow C \rightarrow D$.
- It has 3 hops, which is more than the other two routes.
- This shows that the shortest path in terms of hops is not always the least expensive.

Ans:- In Uniform Cost Search, a node may be reached through different paths with different costs. When the goal node is first generated, there may still be another path in the priority queue that reaches the same goal with a lower total cost.

Since UCS always removes the lowest-cost path first, we can guarantee that the first time the goal node is popped from the priority queue, its path cost is the minimum possible. If we stop as soon as the goal is generated, we might return a more expensive path and miss the optimal solution.

Here:

- Path $A \rightarrow B \rightarrow D$ is generated with cost 6.

- Later, UCS finds $A \rightarrow B \rightarrow C \rightarrow D$ with cost 4.

- If we stop when D is first generated, we would incorrectly return cost 6.

- By waiting until D is popped from the priority queue, UCS correctly returns the optimal cost 4.

Uniform Cost Search guarantees the optimal path by always expanding the least-cost path first.